Actividad: Strings

Programación I, Semana 9

Nombres: _____ y _____

Sección: _____

Esta actividad **no tiene nota**.

Ejercicios obligatorios

Ejercicio 1: Contar una letra

Completar los espacios para que la función cuente cuántas veces aparece el carácter `c` en el string `s`.

Pista: es el mismo contador de la semana 8. Revisar las comillas.

```
int contarLetra(std::string s, char c) {
    int n = _______________;
    int cuenta = _______;
    for (int i = ____; i < ____; i++) {
        if (_____________________) {
            _____________________;
        }
    }
    return __________;
}
```

Resultado esperado: `contarLetra("banana", 'a')` devuelve **3** y `contarLetra("banana", 'n')` devuelve **2**.

Escribir la línea de `main` que imprime cuántas letras `a` tiene `"banana"`:

Ejercicio 2: Seguimiento

```
int buscarLetra(std::string s, char c) {
    int n = s.size();
    int pos = -1;
    for (int i = 0; i < n && pos == -1; i++) {
        if (s[i] == c) {
            pos = i;
        }
    }
    return pos;
}
```

Completar la tabla de seguimiento de `buscarLetra("computador", 't')`.
Resultado esperado: la función devuelve **5**.

`i`	`s[i]`	`¿s[i] == 't'?`	`pos`
inicio			

¿Por qué el ciclo no llega a revisar `s[6]`?

Ejercicio 3: ¿Qué imprime?

a)

```
std::string s = "hola";
s[0] = 'H';
std::cout << s << " " << s.size();
```

Imprime: _____

b)

Pista: recordar qué ocurre con un string que se pasa por valor.

```
void cambiar(std::string s) {
    s[0] = 'X';
}

int main() {
    std::string p = "sol";
    cambiar(p);
    std::cout << p;
    return 0;
}
```

Imprime: _____

c)

```
std::string a = "ab";
std::string b = a + "cd" + a;
std::cout << b << " " << b[3];
```

Imprime: _____

Ejercicio 4: Encontrar los errores

Esta función debería contar las vocales de un string, pero tiene **4 errores**. Encerrar cada uno en un círculo y escribir al lado la corrección.
Resultado esperado: ya corregido, el programa imprime **5**.

```
int contarVocales(std::string s) {
    int n = s.size();
    int c;
    for (int i = 0; i <= n; i++) {
        if (s[i] == 'a' || s[i] == "e" || s[i] == 'i' ||
            s[i] == 'o' || s[i] == 'u') {
            c++;
        }
    }
}

int main() {
    std::cout << contarVocales("murcielago");
    return 0;
}
```

Ejercicios intermedios

Ejercicio 5: Palíndromo

Un palíndromo se lee igual de izquierda a derecha que de derecha a izquierda, como `"oso"` o `"reconocer"`. Escribir **completa** la función `bool esPalindromo(std::string s)`.

> *Pista: comparar el primer carácter con el último, el segundo con el penúltimo, y así sucesivamente. Desde el final, la pareja de* `s[i]` *es* `s[n - 1 - i]`*.*

Resultado esperado: `esPalindromo("reconocer")` devuelve **true** y `esPalindromo("programa")` devuelve **false**.

Reto

Ejercicio 6: Lista de nombres

```
const int N = 5;
std::string nombres[N] = {"Ana", "Luis", "Andres", "Maria", "Alberto"};
```

a) Escribir la función `int contarInicial(std::string nombres[], int n, char letra)`, que cuenta cuántos nombres empiezan con `letra`.

Pista: `nombres[i]` es un string, y `nombres[i][0]` es su primera letra.

Resultado esperado: `contarInicial(nombres, N, 'A')` devuelve **3**.

b) Escribir la función `int buscarNombre(std::string nombres[], int n, std::string x)`, que devuelve la posición de `x` en el arreglo, o `-1` si no está.

Resultado esperado: con `"Maria"` devuelve **3**; con `"Pedro"` devuelve **—1**.